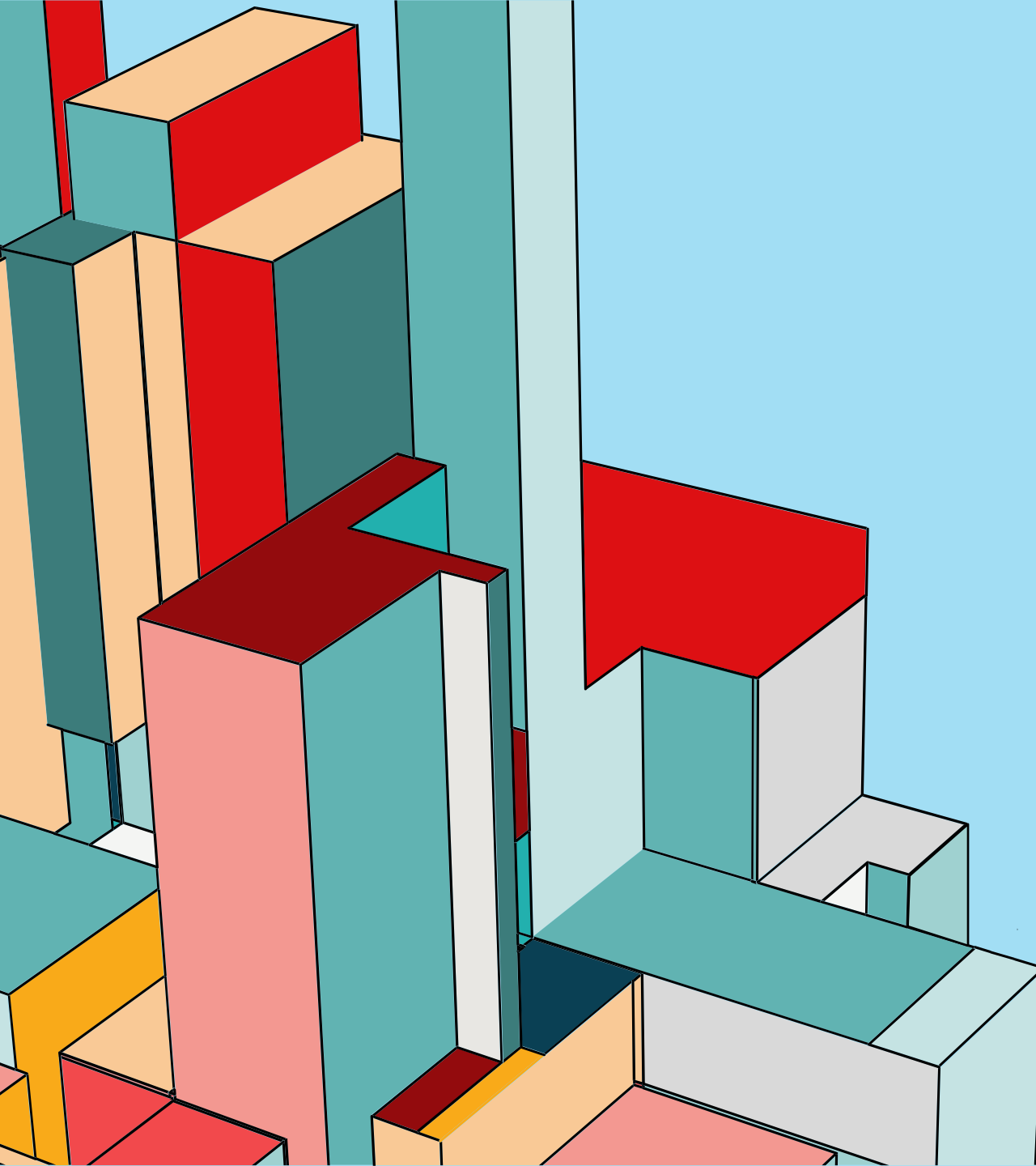


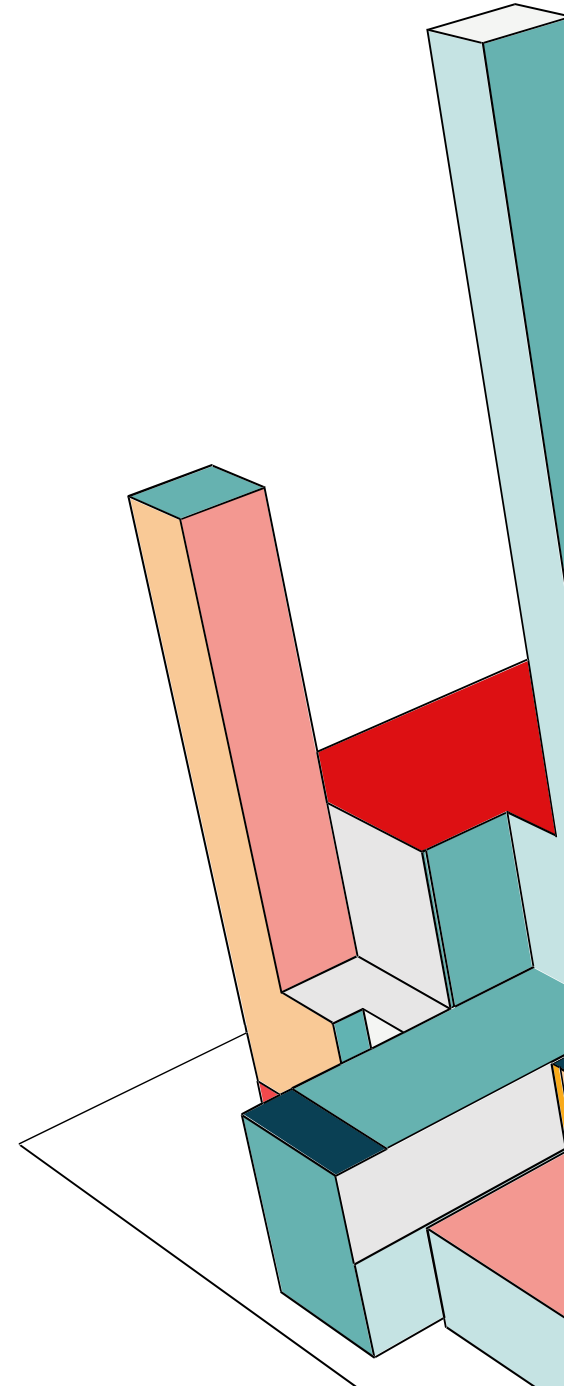


LECTURE 8 **THE $O(N \log N)$ SORTING BOUND**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS

- Exam 1 is on Wed. 2/11, starting at 8am in DCC.
 - Open paper notes (your own + slides/HW/etc.)
- No labs this week
- HW4 due Tuesday as usual; no new HW this week.



DECISION TREES

Have you ever seen the kids' game Guess Who?

The idea is that you are allowed to ask a series of yes/no questions, trying to narrow down which character is the target.

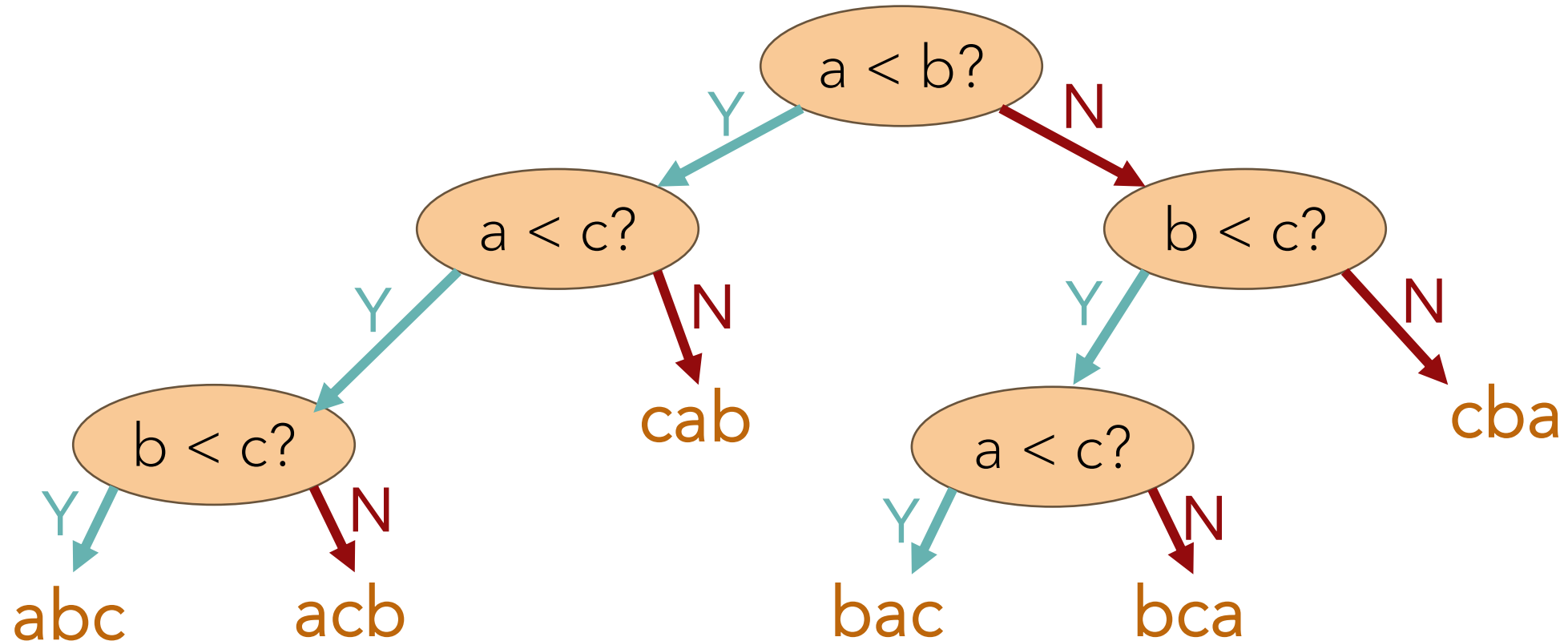
This is the basic premise of decision trees.



Image credit: Joe Stone

DECISION TREES

Comparison-based sorting effectively use a decision tree. Here is a simple example for three elements.



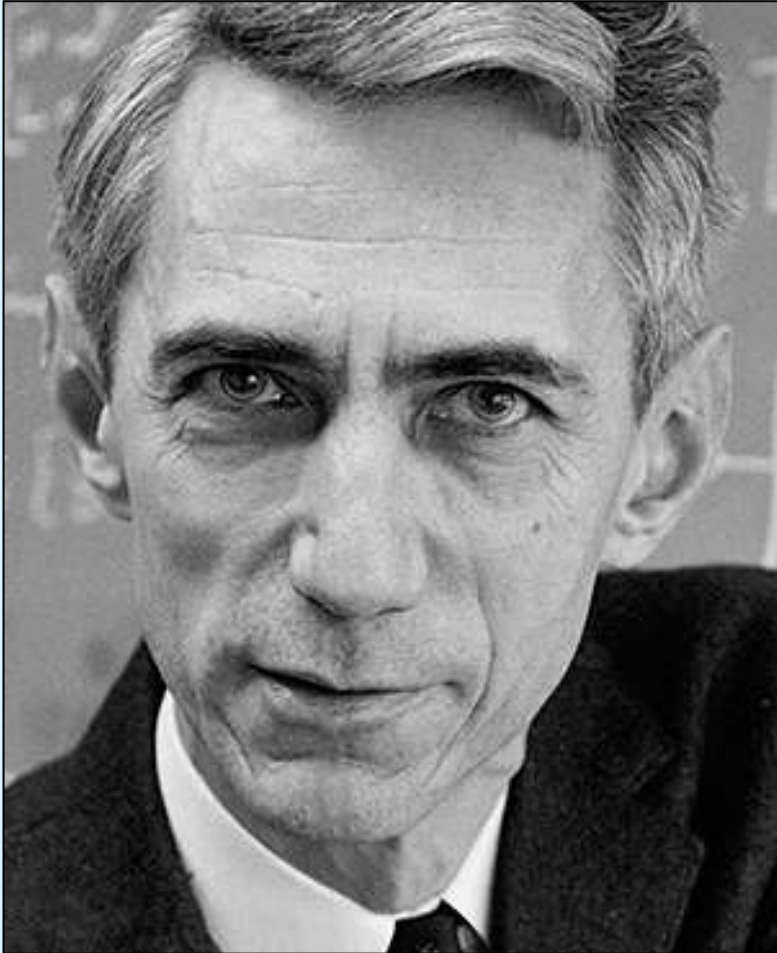
DECISION TREES

Key question: Given a certain number of questions, what is the maximum number of cases one can distinguish among?

Answering questions like this puts us in the realm of *information theory*...

... which gives me an excuse to take a few minutes to talk about **my boi**...

THE FATHER OF INFORMATION THEORY



One could be forgiven for mistaking him for the actor Peter Cushing, but this is in fact [Claude Shannon](#), the most important computer scientist you've probably never heard of.

"John von Neumann, Alan Turing and many other visionaries gave us computers that could process information. But it was Claude Shannon who gave us the modern concept of information" -MIT Technology Review

I highly recommend the documentary about his life, [The Bit Player](#).



QUICK BIO

Started college at Michigan at age 16; graduated with two degrees (EE & math)

Grad school at MIT – worked with **Vannevar Bush**

Master's thesis proved Boolean algebra and digital circuits were equivalent systems

Finished Ph.D. at 24 ("An Algebra for Theoretical Genetics")

Went to Institute for Advanced Study in Princeton

Hung out with **John von Neumann**, **Albert Einstein**, **Kurt Gödel**.

During WWII, worked at Bell Labs on cryptography and fire-control systems, and met **Alan Turing**.

MIT faculty from 1956-1978.

Developed Alzheimer's; died in 2001 after a long illness.

PRANKSTER, JUGGLER, GADGETEER



Shannon was a prolific inventor, an inveterate prankster, and one of the most creative people the field has ever produced. Seen here are many of his toys and gadgets. (He also frequently arrived at class on a unicycle.



“A MATHEMATICAL THEORY OF COMMUNICATION”

This paper was published in the *Bell Systems Technical Journal* in 1948, combining Shannon's love of probability and physics with his work at Bell on cryptography. It ties the physics concept of *entropy* to information – the amount of disorder or uncertainty available in a system is the amount of information it contains.

- One colleague later compared it to Einstein's theory of relativity and Heisenberg's uncertainty principle.
- Another summed it up this way: “It's like saying how much influence the inventor of the alphabet had on literature.”

IMPACTS OF INFORMATION THEORY

Literally anything that has to do with storing, sending, or manipulating information!

- Communications
- Cryptography
- Data compression
- Error correcting codes
- Gambling
- Game theory
- Bioinformatics
- Physics / black holes
- ...

THE IDEA WE NEED HERE

Every comparison we do during a sort provides us with one bit of information.

To differentiate among k choices at the bottom of the decision tree, we must ask at least $h = \lceil \log_2 k \rceil$ questions. Information theory guarantees us that we can do no better.

h is the height of the minimal decision tree with k leaves.

A LITTLE HELP FROM STIRLING'S FORMULA

The decision tree for a sort of n elements must distinguish between $n!$ different orderings – that is, the decision tree will have $n!$ leaves. Thus, $h \geq \log_2 n!$.

$\log_2 n!$ is annoying, but *Stirling's formula* says $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$.

$$\log_2 \sqrt{2\pi n} \left(\frac{n}{e}\right)^n = n \lg n - n \lg e + \frac{1}{2} \lg n + \frac{1}{2} \lg 2\pi = \Theta(n \log n)$$

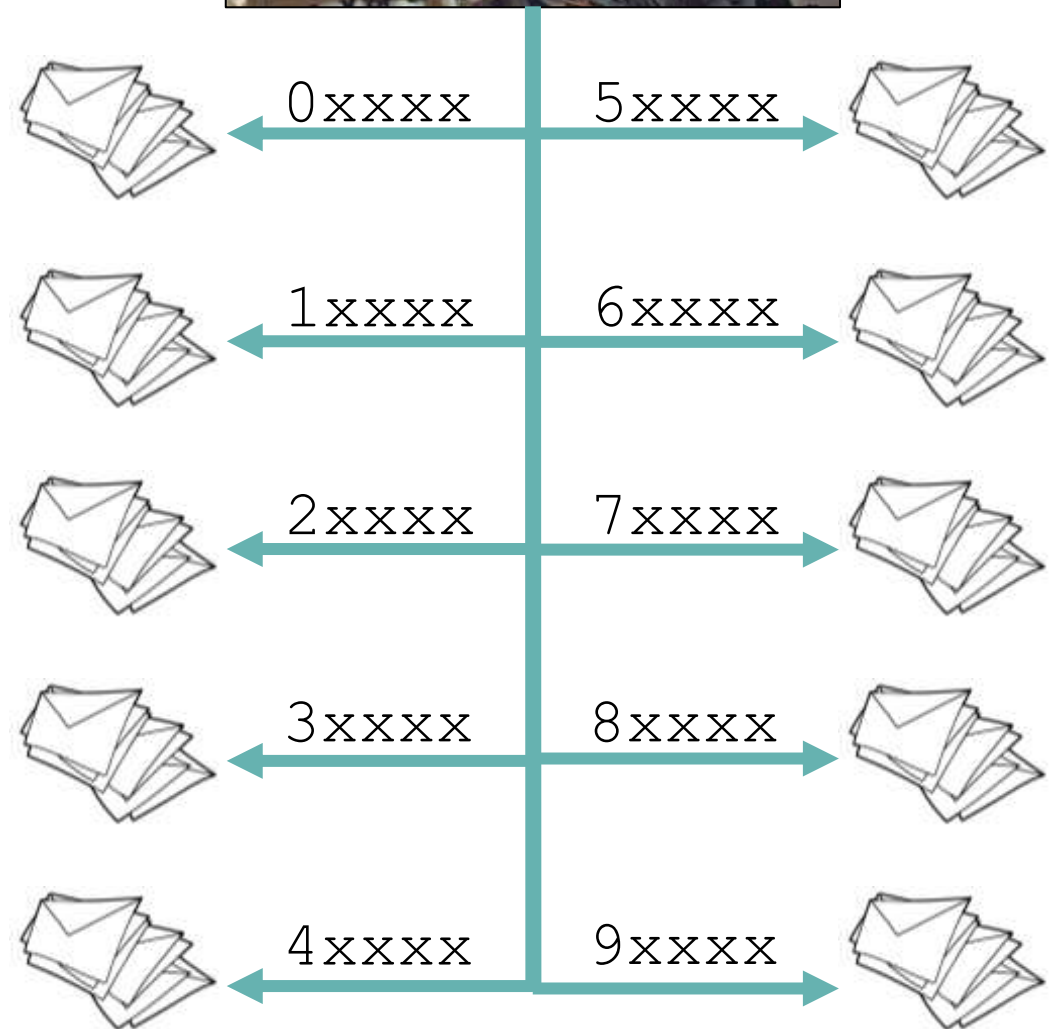
Thus, the asymptotic lower bound for comparison-based sorting is $n \log n$. Since we have sorts that achieve this bound (mergesort, quicksort, heapsort), this bound is *tight* - we cannot improve upon it any further.

OTHER OPTIONS?

We keep saying “comparison-based” sort. What other options are there?

Consider a distribution center for the US Postal Service. It takes in literal tons of mail. How does it organize this?

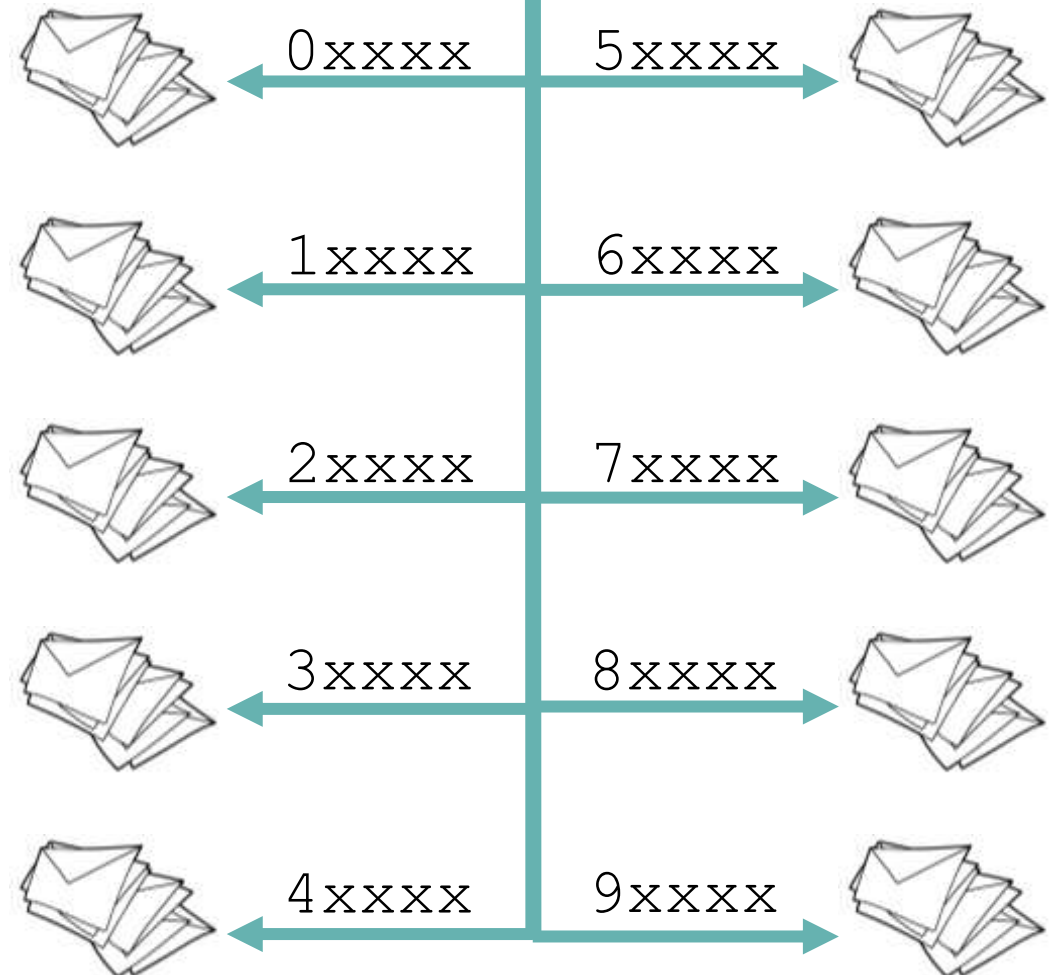
By ZIP code!



OTHER OPTIONS?

Once we have sorted into these ten piles, now what?

Do it again based on the second digit!



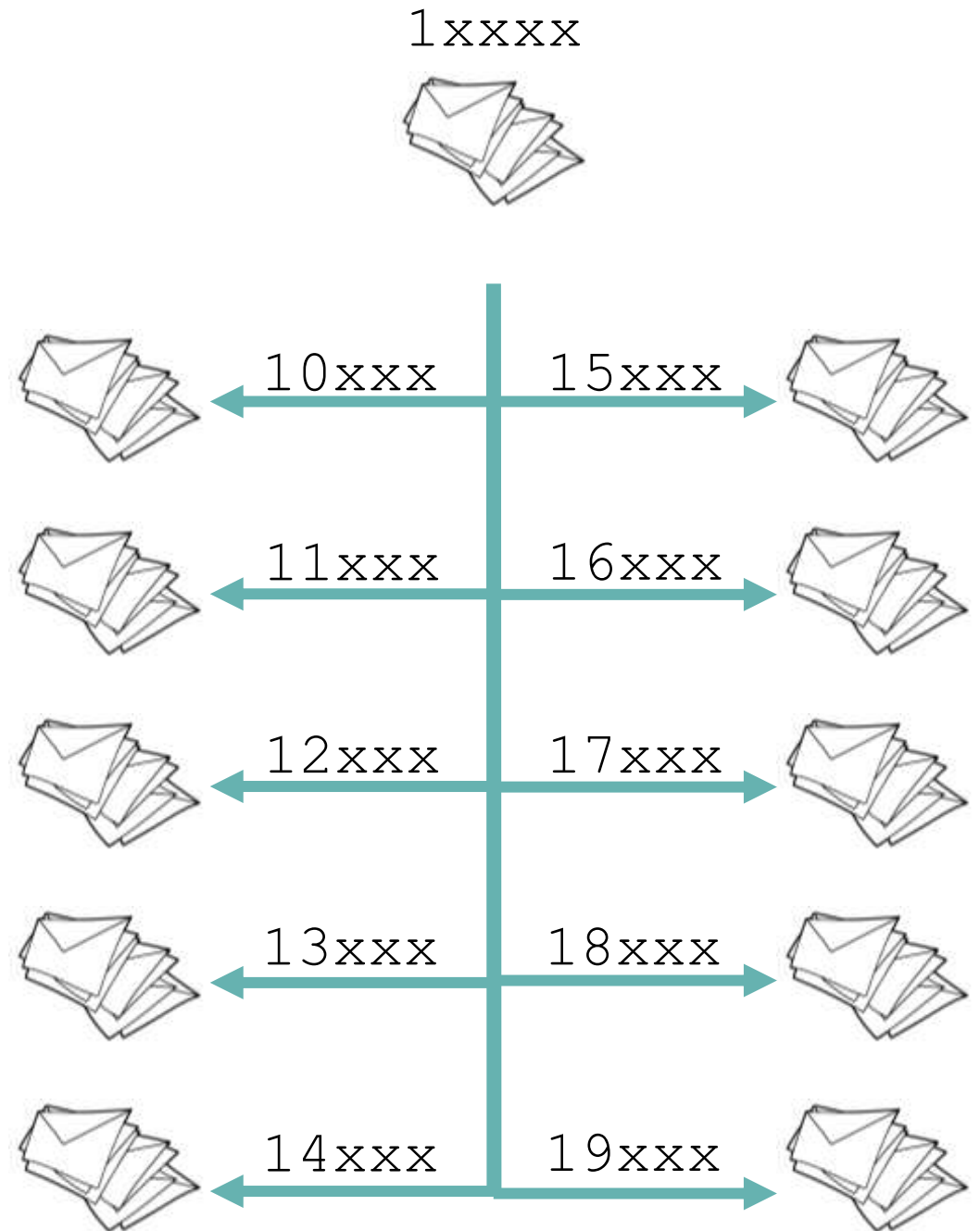
OTHER OPTIONS?

Once we have sorted into these ten piles, now what?

Do it again based on the second digit!

And then the third, fourth, and fifth digits.

What is the runtime of this sorting algorithm?



SUBMITTY POLL

What is the runtime for the ZIP code sorting algorithm?



RADIX SORT

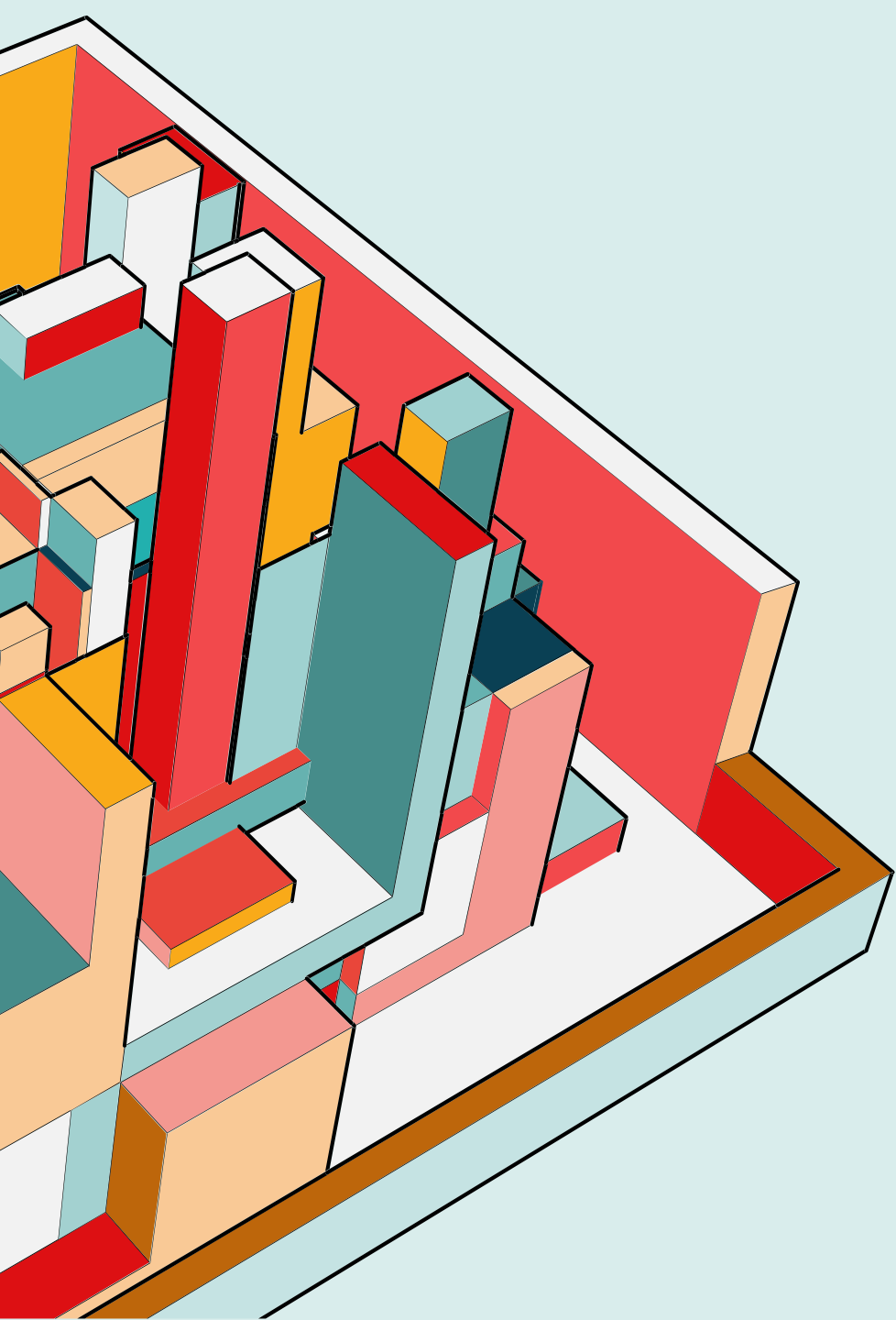
This approach (common anywhere that fixed-length strings are the sort keys) is known as *radix sort* or *bucket sort*.

Observe that at no point do we compare pairs of sort keys against each other; thus, the $\Theta(n \log n)$ bound does not apply.

Radix sort will make a single pass through all n elements for each digit in the sort keys. Thus, its runtime is $\Theta(nd)$.

Observe that, if our n elements have mostly distinct keys, then d will need to be at least $\log n$ for us to have enough unique keys. So unless we have LOTS of duplication, radix sort doesn't actually buy us anything asymptotically.

17 It also requires extra memory to hold the "buckets".



EXAM I REVIEW

EXAM DETAILS

Wednesday starting at 8am in the DCC:

- Accommodations room: DCC 232
- Otherwise, last names:
 - A-G: DCC 324
 - H-L: DCC 337
 - M-Z: DCC 318

Please bring your ID to the exam. Your paper will not be accepted until a TA/mentor has verified your ID.

GROUND RULES

The default time for the exam is 110 minutes.

You may use paper notes, including printouts, in any quantity. However, these must be either your own notes or materials I have provided (slides, HW, textbooks).

- DO NOT print out someone else's notes.
- Reminder that use of ChatGPT or similar, even to summarize notes for you, is considered academic dishonesty.

Obviously, any collaboration or other communication with anyone other than course staff during the exam is strictly forbidden.

LECTURE 1 – WRITING ALGORITHMS

Make sure your algorithms are unambiguous...

... without getting bogged down too much in the details! Feel free to “call” simpler algorithms, etc.

In general, it’s best if you can describe your algorithm in English.

Pseudocode is fine, but make sure it’s pseudocode – don’t try to write actual code. (For example, avoid getting caught up in loop increments, precise array indices, etc.)

LECTURE 2 – ASYMPTOTICS

Understand & use all five asymptotic notations.

Proofs:

- For O, Ω, Θ – specify the constants!
- For o, ω – remember that you must find an n_0 for every positive c ... which means n_0 will have to be a function of c .
 - Or you can use limits.

Non-recursive runtimes: Look for loops; summations

Recursive runtimes: Write a recurrence relation & solve

LECTURE 2 – ASYMPTOTICS

Understand & use all five asymptotic notations.

Proofs:

- For O, Ω, Θ – specify the constants!
- For o, ω – remember that you must find an n_0 for every positive c ... which means n_0 will have to be a function of c .
 - Or you can use limits.

Non-recursive runtimes: Look for loops; summations

Recursive runtimes: Write a recurrence relation & solve

LECTURE 3 – BRUTE FORCE

If all else fails, try everything!

Actually, most of the time your first attempt should be the naive / brute-force approach – it sets an upper bound (not necessarily tight) on runtime for any reasonable algorithm.

Brute force solutions often have simple enhancements that provide some speed up (adjusting the constants hidden by big-O)

Selection sort & bubble sort

LECTURE 4 – SEARCHES; INDUCTIVE I

BFS & DFS – critical algorithms

- Execution, including back/cross edges
 - Trees
 - Visit or in/out times
- Analysis – what is their runtime and why?

Insertion sort

- Execution & analysis

LECTURE 5 – INDUCTIVE II

General idea: “decrease and conquer”

- Reduce problem size and solve smaller problem recursively
 - Constant size reduction (insertion sort, permutations)
 - Constant factor reduction (binary search, RPM)
 - Variable reduction (Euclid’s GCD)
- Key point: Only have to process one smaller problem

All of these will generally involve writing / solving a recurrence relation.

LECTURE 6 – DIVIDE & CONQUER I

Quickselect

- Lomuto partition & overall algorithm
- Recurrence relation & runtime analysis

Mergesort

- Execution details
- Recurrence relation & runtime analysis

Master Theorem

- Use to solve D&C recurrences

LECTURE 7 – DIVIDE & CONQUER II

Quicksort (including Hoare partition)

- Operation and runtime analysis

Karatsuba integer multiplication

Closest pair problem

- Both of these rely on a hybrid approach – they pull some of the work into the non-recursive part of the algorithm so that the recursive tree is not quite as wide, producing a better overall asymptotic runtime.

BEFORE YOU LEAVE

